

Predictive Assignment PS-5

Shreyanshi Adak - 745

2026-02-26

K nearest neighbours regression

1. Problem to demonstrate the utility of K nearest neighbour regression over least squares regression :

Consider a setting with $n = 1000$ observations. Generate,

- i. x_{1i} from $N(0, 22)$ and x_{2i} from $\text{Poisson}(\lambda = 1.5)$.
- ii. ϵ_i from $N(0, 1)$.
- iii. $y_i = -2 + 1.4x_{1i} - 2.6x_{2i} + \epsilon_i$.

Divide the dataset into two parts by assigning the first 800 observations to the training set and using all the remaining observations as the testing set.

```
rm(list=ls())
set.seed(123)
n=1000
x1=rnorm(n,0,2)
x2=rpois(n,1.5)
e=rnorm(n)
y=-2+1.4*x1-2.6*x2+e
train.data=data.frame(y[1:800],x1[1:800],x2[1:800])
colnames(train.data)=c("y","x1","x2")
test.data=data.frame(y[801:1000],x1[801:1000],x2[801:1000])
colnames(test.data)=c("y","x1","x2")
```

1. Fit a multiple linear regression equation of y on x_1 and x_2 . Calculate test MSE.

```
fit1=lm(y~x1+x2,data=train.data)
summary(fit1)

##
## Call:
## lm(formula = y ~ x1 + x2, data = train.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.0727 -0.6573 -0.0125  0.6921  3.2412
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.07300    0.05382  -38.52  <2e-16 ***
## x1           1.38207    0.01767   78.21  <2e-16 ***
```

```
## x2          -2.55584    0.02768  -92.34   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.98 on 797 degrees of freedom
## Multiple R-squared:  0.9492, Adjusted R-squared:  0.9491
## F-statistic: 7445 on 2 and 797 DF,  p-value: < 2.2e-16

y.hat.test=predict(fit1,newdata=test.data)
mse.test=mean((test.data$y-y.hat.test)^2)
mse.test

## [1] 0.998901
```

2. Fit a KNN model with $k = 1, 2, 5, 9, 15$. Calculate test MSE for each choice of k .

```
k=c(1,2,5,9,15)
library(FNN)

## Warning: package 'FNN' was built under R version 4.5.2

X.train=matrix(c(x1[1:800],x2[1:800]),byrow=F,ncol=2)
X.test=matrix(c(x1[801:1000],x2[801:1000]),byrow=F,ncol=2)
f=function(k){
  pred=knn.reg(train=X.train,test=X.test,y=train.data$y,k)
  mean((test.data$y-pred$pred)^2)
}
f(1);f(2);f(5);f(9);f(15)

## [1] 2.219793
## [1] 1.729587
## [1] 1.303978
## [1] 1.205371
## [1] 1.235776

library(caret)

## Warning: package 'caret' was built under R version 4.5.2
## Loading required package: ggplot2
## Warning: package 'ggplot2' was built under R version 4.5.2
## Loading required package: lattice

tr_kr_1=knnreg(y~x1+x2,data=train.data,k=1)
y_KNN_1=predict(tr_kr_1,test.data)
mse_knn_1=mean((test.data$y-y_KNN_1)^2)
mse_knn_1
```

```
## [1] 2.219793
```

Suppose the data in Step (iii) is generated as :

$$y_i = 1/(-2 + 1.4x_{1i} - 2.6x_{2i} + 2.9x_{1i}x_{2i}) + 3.1 \sin(x_{2i}) - 1.5x_{1i}x_{2i}^2 + \epsilon_i.$$

Work out the problems in (1) and (2). Compare and comment on the results.

```
y.new=1/(-2+1.4*x1-2.6*x2+2.9*x1*x1)+3.1*sin(x2)-1.5*x1*x2*x2+e
train.data.new=data.frame(y.new[1:800],x1[1:800],x2[1:800])
colnames(train.data.new)=c("y", "x1", "x2")
test.data.new=data.frame(y.new[801:1000],x1[801:1000],x2[801:1000])
colnames(test.data.new)=c("y", "x1", "x2")
fit3=lm(y~x1+x2,data=train.data.new)
summary(fit3)

##
## Call:
## lm(formula = y ~ x1 + x2, data = train.data.new)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -241.819   -5.150    -0.051     5.566   155.662
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.2369      1.0272  -0.231  0.81764
## x1           -6.3747      0.3373 -18.901 < 2e-16 ***
## x2            1.3849      0.5283   2.621  0.00892 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 18.7 on 797 degrees of freedom
## Multiple R-squared:  0.3146, Adjusted R-squared:  0.3129
## F-statistic: 182.9 on 2 and 797 DF,  p-value: < 2.2e-16

y.hat.test.new=predict(fit3,newdata=test.data.new)
mse.test.new=mean((test.data.new$y-y.hat.test.new)^2)
mse.test.new

## [1] 205.1776
```

Case 1: When the Underlying Pattern is Linear :

In this scenario, the dependent variable was created using a linear equation. Since multiple linear regression is specifically built to estimate linear relationships, it is naturally well-suited for this situation.

Here we observe:

Linear regression achieves the smallest (or nearly the smallest) test Mean Squared Error (MSE).

KNN regression results in a comparatively larger test MSE.

Although increasing the value of k in KNN makes the model more stable, it still fails to outperform linear regression.

Explanation:

Because the true relationship between predictors and response is linear, linear regression is able to accurately represent the actual structure of the data. In contrast, KNN is a non-parametric technique that makes predictions based only on nearby data points. It does not explicitly use the overall linear pattern present in the dataset. As a result, linear regression provides more accurate predictions in this case.

Final remark ->

When the real data-generating process follows a linear form, linear regression is the more appropriate and effective method.

Case 2: When the Relationship is Nonlinear :

In this case, the response variable was generated from a nonlinear function. The model included:

A squared (quadratic) term

A sine component

An interaction term

Such nonlinear elements cannot be properly represented by a basic linear regression model.

Here we observe,

Linear regression shows a very large test MSE.

KNN regression produces a much smaller test MSE.

Intermediate values of k (for example, 5, 9, or 15) tend to provide the best performance.

Explanation:

Linear regression assumes that the relationship between predictors and response is linear. Because of this assumption, it struggles to capture complex nonlinear patterns, leading to poor predictive accuracy.

KNN regression, on the other hand, does not impose any specific functional form. It makes predictions using nearby observations, allowing it to naturally adjust to nonlinear structures in the data.

Final remark ->

When the true relationship is nonlinear, KNN regression clearly outperforms linear regression.